4-bit Array Multiplier: RTL to GDSII on SKY130

Digital VLSI Design — Assignment Report

Srivarshitha Medarametla (2022102030) Sasidhar Varada (2024702015)

Technology: SkyWater SKY130, library `sky130_fd_sc_hd`, PDK version 1689ac3f. **Tools:** Icarus Verilog, GTKWave, Yosys, OpenSTA, LibreLane (OpenROAD, Magic, KLayout, Netgen) in the IIC-OSIC-TOOLS Docker image. All commands are listed in the repository `README.md`.

Repository: github.com/16srivarshitha/array_multiplier_rtl_to_gdsii

1 Part A — RTL Design and Verification

1.1 Simulation results

RTL simulation of all 256 input combinations:

```
PASS: 256 vectors checked, 0 errors
```

Error test (the `sum` output in `full_adder.v` temporarily changed to `a ^ b`):

```
FAIL: 256 vectors checked, 95 errors
```

The change was then undone, and the testbench passed again.

1.2 Hand-worked example

Roll number used: 2022102030 (Sri Varshitha Medarametla), last digit 0. $A = 0 + 5 = 5 = 0101_2$ and $B = 7 = 0111_2$.

```
          0 1 0 1   A
      ×   0 1 1 1   B
-------------------------------------------
          0 1 0 1   pp0 = A AND b0 (b0 = 1)
        0 1 0 1     pp1 = A AND b1 (b1 = 1), shifted 1
      0 1 0 1       pp2 = A AND b2 (b2 = 1), shifted 2
    0 0 0 0         pp3 = A AND b3 (b3 = 0), shifted 3
-------------------------------------------
0 0 1 0 0 0 1 1   P = 35
```

Check: $5 + 10 + 20 + 0 = 35 = 00100011_2 = 0\text{x}23$. In the waveform, $a = 5$, $b = 7$ must give $p = 0\text{x}23$ two clock edges later.

1.3 Waveform

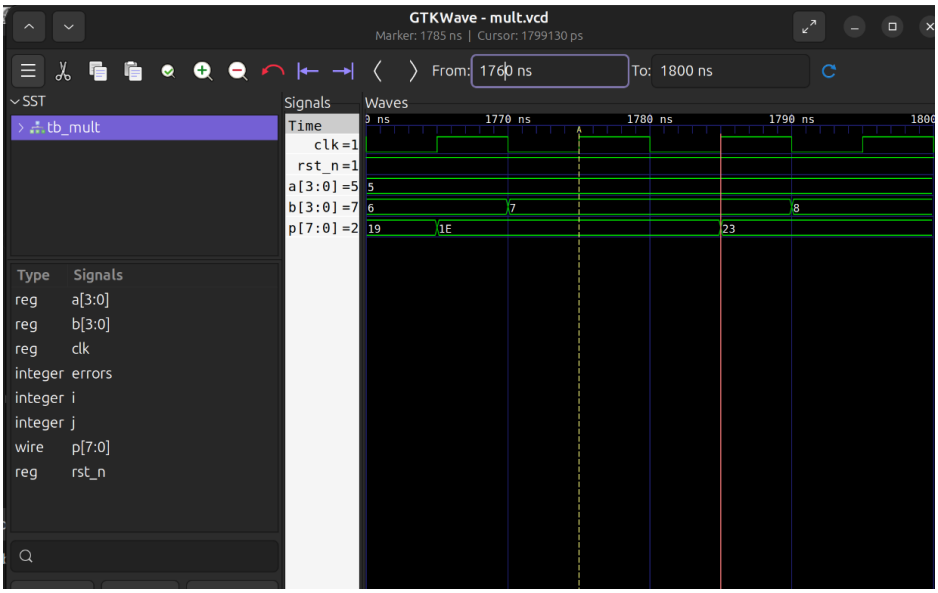


Figure 1: GTKWave screenshot for $A = 5$, $B = 7$ showing `clk`, `rst_n`, `a`, `b` and `p`. The 2-cycle latency (inputs captured at edge 1, product registered at edge 2) is marked.

1.4 Block diagram

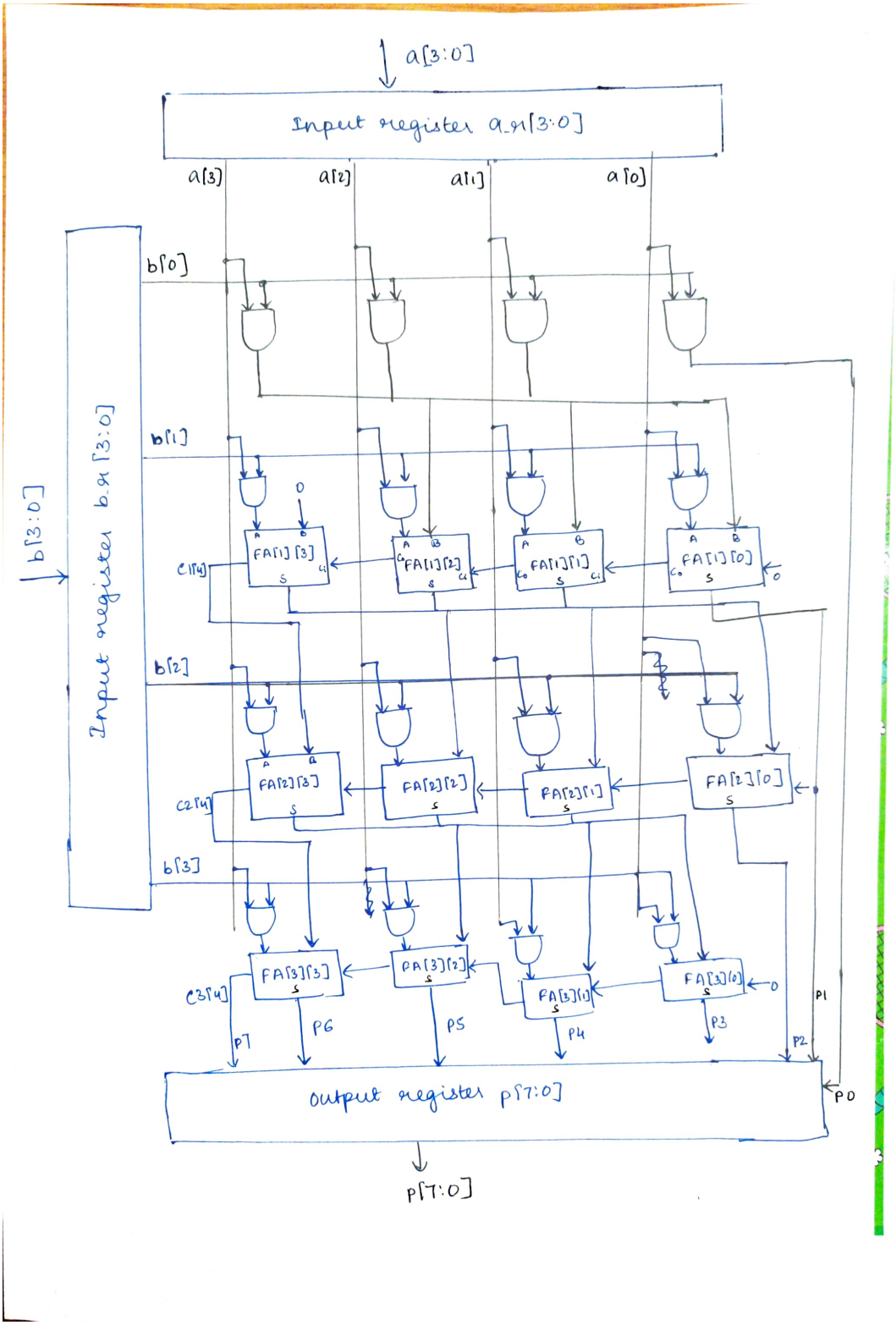


Figure 2: Block diagram: input registers `a_r`, `b_r`, 16 AND gates, 12 full adders (rows 1–3) and the output register `p`.

2 Part B — Logic Synthesis (Yosys)

Table 1: Synthesis results from `mult_array_stat.rpt`.

Metric	Value
Total number of cells	67
Number of flip-flops	16 (`dfrtp_1`)
Total cell area (μm^2)	784.502
Flip-flop area (μm^2)	400.384 (16×25.024)
Flip-flop area (% of total)	51.04%
Three most-used cell types	`dfrtp_1` (16), `nand2_1` (9), `xnor2_1` (7)
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

Other figures from the same report: combinational logic is 51 cells with area 384.12 μm^2. The logic includes 5 `maj3_1`, 3 three-input XOR/XNOR cells (2 `xnor3_1`, 1 `xor3_1`) and 7 `xnor2_1`.

The testbench check was moved from 1 ns to 2 ns after the clock edge. In the gate-level run the flip-flops also update 1 ns after the edge (`UNIT_DELAY=#1`), so the earlier check read every result one vector late.

3 Part C — Static Timing Analysis (OpenSTA)

Constraints as specified: typical corner `tt_025C_1v80`, 1.0 ns input and output delays, ideal clock.

Table 2: Part C results (typical corner).

Metric	Value
Worst slack at $T = 10$ ns	7.020 ns
$T_{min} = 10 - 7.020$	2.98 ns
$F_{max} = 1000/2.98$	335.6 MHz
Slack at $T = T_{min}$	≈ 0 ns
Critical path start point	`_106_` (input register bit `b_r[0]`)
Critical path end point	`_100_` (output register, bit `p[6]`)
Cells on the critical path	8 combinational (9 including the launch flip-flop)
Latency $= 2 \times T_{min}$	5.96 ns
Throughput	335.6 million results/s

Cells on the critical path, in order

#	Instance (cell)	Delay (ns)	Role
0	_106_ (dfrtp_1)	0.411	launch flip-flop, clk-to-Q
1	_048_ (and4_1)	0.218	partial-product logic
2	_050_ (a2111oi_0)	0.428	partial-product / row logic
3	_051_ (a21oi_1)	0.094	row logic
4	_053_ (mux2i_1)	0.218	row logic
5	_069_ (xnor3_1)	0.374	full-adder sum
6	_074_ (maj3_1)	0.371	full-adder carry
7	_076_ (maj3_1)	0.376	full-adder carry
8	_080_ (xor3_1)	0.355	full-adder sum

Data arrival time 2.844 ns; library setup time 0.136 ns; required time 9.864 ns; slack +7.020 ns. Four of the eight combinational cells (xnor3, two maj3, xor3) are the sum and carry cells of full adders.

3.1 Additional analysis: stricter constraints, three corners

Run with a 10 ns clock, 0.25 ns clock uncertainty and 2 ns input/output delays (our own choice, stricter than the assignment). Pre-layout, ideal clock, no parasitics. All setup and hold checks pass.

Corner	Setup slack (ns)	Hold slack (ns)	Min period (ns)	F_{max} (MHz)
ss_100C_1v60	3.433	0.807	6.57	152
tt_025C_1v80	6.770	0.275	3.23	310
ff_n40C_1v95	7.346	0.078	1.92	522

Setup is worst at the slow corner and hold is worst at the fast corner. The smallest hold margin (0.078 ns, fast corner) belongs to the shortest path, a[0] AND b[0] into p[0].

4 Part D — Physical Design (LibreLane)

Configuration: DESIGN_NAME = "mult_array", CLOCK_PORT = "clk", FP_SIZING = "absolute", DIE_AREA = [0, 0, 150, 150]. The starting CLOCK_PERIOD was $1.2 \times 2.98 = 3.58 \rightarrow 4.0$ ns, raised by 0.5 ns after each run that failed timing at the slow corner (ss_100C_1v60).

Table 3: Every attempt.

Run	CLOCK_PERIOD (ns)	Worst setup slack (ns)
1	4.0	negative (FAIL)
2	4.5	negative (FAIL)
3	5.0	−0.957 (FAIL)
4	5.5	−0.456 (FAIL)
5	6.0	−0.128 (FAIL)
6	6.5	−0.157 (FAIL)
7	7.0	+0.076 (PASS)

The first attempts accidentally used the container's default PDK (IHP SG13G2). They were discarded and the flow was re-run with `--pdk sky130A`.

Table 4: Part D results (final run).

Metric	Value
`CLOCK_PERIOD` used	7.0 ns
Number of runs needed	7
Core area	17,759.5 μm^2
Cells after synthesis (Part B)	67
Cells after place and route	1,901
Kinds of cells added	tap cells, fill cells, clock buffers, timing-repair buffers
Wirelength	2,340 μm
Setup slack	+0.076 ns
Post-route $F_{max} = 1000/(7.0 - 0.076)$	144.4 MHz
F_{max} from Part C	335.6 MHz (tt, ideal); 152 MHz (ss, stricter constraints)
DRC errors	0
LVS errors	0

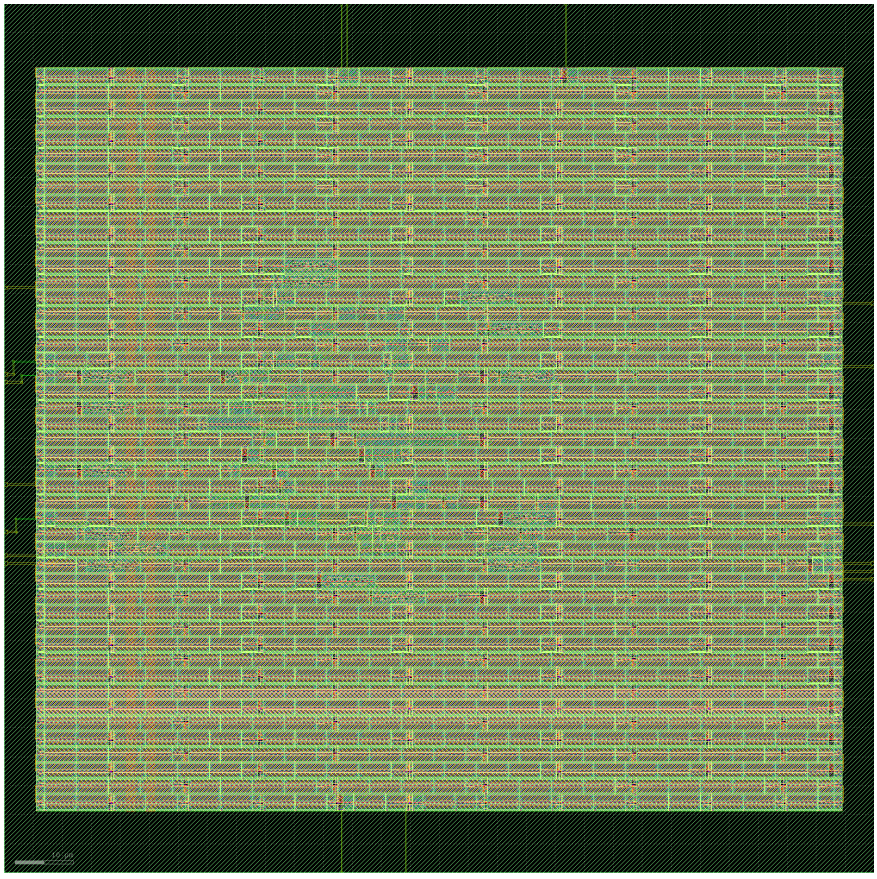


Figure 3: Final layout, whole die (KLayout).

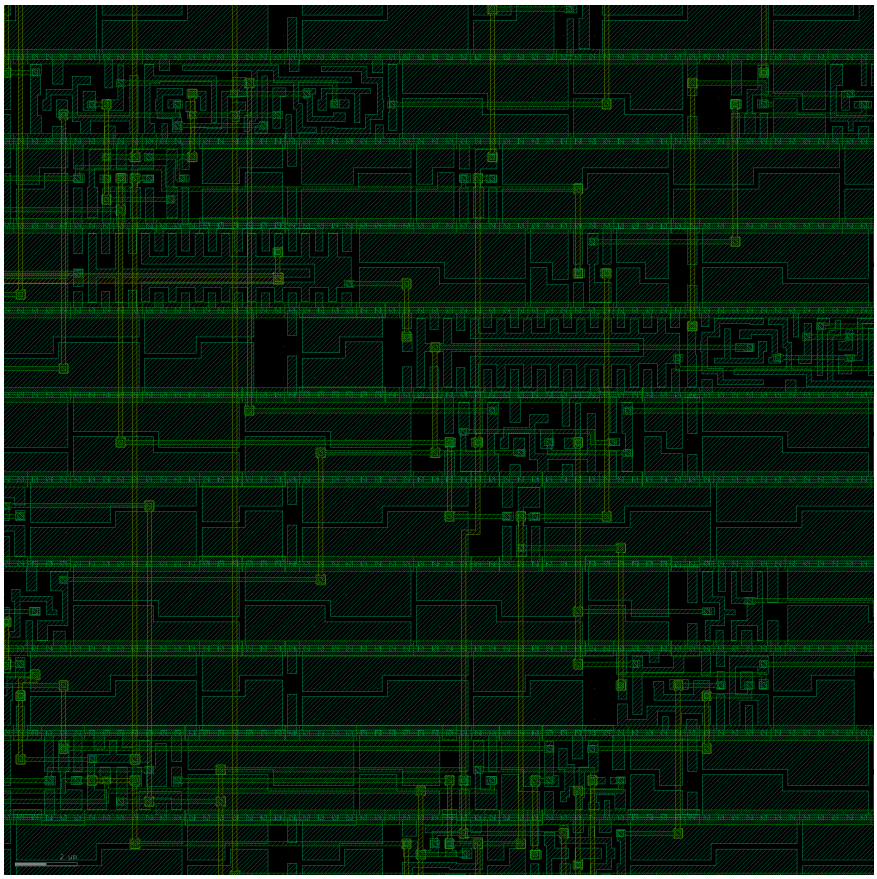


Figure 4: Zoomed view: standard cell rows and metal wires.

5 Questions

1. Cells for the logic. Synthesis produced $67 - 16 = 51$ cells for the logic, not $16 + 12 = 28$. Yosys flattens the hierarchy and ABC maps the whole design as one network of library gates, so the RTL blocks do not survive as units. A full adder has two outputs (sum and carry), and no single full-adder cell appears in the netlist. Instead the sum is built from `xor3`/`xnor3` or `xnor2` cells and the carry from `maj3` or NAND/AOI gates (the netlist has 5 `maj3`, 3 three-input XOR cells and 7 `xnor2`). The three adders with a constant-0 carry-in (the first one in each row) reduce to half adders, which ABC simplifies further. ABC also merges many of the 16 AND gates into other cells: only 4 `and2` and 2 `and4` remain as separate ANDs, while the rest appear as 9 `nand2` cells or inside complex gates such as `a2111oi` and `a22oi`. Finally, ABC adds inverters and a few small low-power-library cells wherever inverted logic is cheaper. The result is a global optimisation of area and delay, not a one-to-one mapping of the RTL.

2. Critical path. The critical path runs from the input register holding `b[0]` (`_106_`) to the output-register bit `p[6]` (`_100_`). It contains 9 cells in the report: the launching flip-flop (0.411 ns clk-to-Q) and 8 combinational cells, followed by a 0.136 ns setup time at the capturing flip-flop. The combinational delay is 2.433 ns, and the data arrives at 2.844 ns, which gives $T_{min} = 2.844 + 0.136 = 2.98$ ns. The estimate of about $2N = 8$ full-adder delays assumes each full adder is one delay on a ripple path. Here only four of the eight cells (`xnor3`, two `maj3`, `xor3`) belong to the adder chain. They take 1.476 ns, about 0.37 ns each, so the logic delay of 2.433 ns equals about 6.6 adder-cell delays instead of 8. The other four cells (`and4`, `a2111oi`, `a21oi`, `mux2i`) are the partial-product ANDs and row logic merged by ABC, so the path is shorter than the 8-FA estimate. It still has the ripple-carry shape the estimate assumes: carry cells chained one after

another.

3. Flip-flop share of area. Flip-flops take 400.384 of 784.502 μm^2, i.e. 51.04%. They are only 16 of 67 cells, but each `dfrtp_1` is 25.02 μm^2, which is about 6.7 times a `nand2_1` (3.75 μm^2) and against an average of about 7.5 μm^2 for the 51 logic cells (384.1 μm^2 in total). The logic is small, since a 4-bit multiplier needs only 16 ANDs and 12 adders. The 16 flip-flops (4+4 input bits, 8 output bits) are a fixed cost of registering all inputs and outputs, and the asynchronous reset makes each one larger than a plain flip-flop. The share would shrink for wider multipliers, since logic grows as N^2 while registers grow only as $4N$.

4. Post-route F_{max}. Post-route F_{max} is 144.4 MHz against 335.6 MHz in Part C. Several effects add up. (i) Part C used the typical corner, while the place-and-route setup slack is reported at the worst corner, the slow one (`ss_100C_1v60`); our own pre-layout slow-corner STA already gave only 152 MHz. (ii) Part C assumed ideal wires, while Part D extracts real wire resistance and capacitance (2,340 μm of wire). (iii) Part C had an ideal clock, while Part D has a real clock tree with skew (about 0.26 ns) and a clock-uncertainty margin. At the typical corner the minimum period goes from 2.98 ns (Part C) to 3.48 ns after routing (287 MHz), and at the slow corner from 6.57 ns before layout to 6.92 ns after (152 to 144.4 MHz). The corner accounts for most of the gap; wires and the clock tree explain the remaining drop of about 5%. The flow added 1,834 cells: 245 *tap cells*, which tie the n-well to VDD and the substrate to GND to prevent latch-up; 1,305 *decap cells* and 260 *fill cells*, which fill the empty parts of the rows to keep the wells and power rails continuous and meet density rules, the decap cells also adding capacitance between VDD and GND to reduce supply noise; 3 *clock buffers*, which build the clock tree so all 16 flip-flops see the clock at nearly the same time; and 20 *timing-repair and port buffers*, which fix slew, long-wire delay and setup violations. No antenna diodes were needed. Only the buffers and the real wires change timing. The added cells explain the growth from 67 to 1,901 cells, since the 150 $\times$ 150 μm die is mostly empty and is filled with tap, decap and fill cells.

5. DRC and LVS. DRC (design rule check) verifies that the drawn geometry obeys the foundry's manufacturing rules: minimum width, spacing, enclosure, minimum area and density of every layer. A layout that fails DRC may not be printable or may yield badly. LVS (layout versus schematic) extracts the transistors and connections from the layout and compares them with the synthesized netlist. It catches shorts, opens and wrong or missing connections and devices. A layout can be manufacturable and still compute the wrong function, or be logically right but unmanufacturable, and a fabricated chip cannot be corrected afterwards. Both must therefore pass before tape-out. Our final run has 0 DRC errors (route, Magic and KLayout) and 0 LVS errors.

6. Scaling to 8 bits. Full adders: $N(N-1) = 8 \times 7 = 56$, against $4 \times 3 = 12$ for $N = 4$ (4.7$\times$). AND gates: $N^2 = 64$, against 16 (4$\times$). Flip-flops: $4N = 32$, against 16. Critical path: about $2N = 16$ full-adder delays against 8, so the logic delay doubles. Using our Part C numbers, the logic delay of 2.433 ns becomes about 4.87 ns. The flip-flop overhead stays fixed at about $0.411 + 0.136 = 0.547$ ns. So $T_{min} \approx 4.87 + 0.55 = 5.41$ ns and $F_{max} \approx 185$ MHz, roughly 45% lower than 335.6 MHz. For area, scaling the 384 μm^2 of logic by $(64 + 56)/(16 + 12) = 4.29$ gives about 1,646 μm^2, plus $32 \times 25.02 = 801$ μm^2 of flip-flops, about 2,447 μm^2 in total (3.1$\times$ the 4-bit design). The flip-flop share would drop from 51% to about 33%. Area grows quadratically ($\sim N^2$) while delay grows only linearly ($\sim N$).